

MADRE

Agentic
System

• MODEL-AGNOSTIC • DELAYED • REASONING • EFFORT • [Beta v0.1]

Product Vision And Research Foundation

Scvria Software
June 20, 2026

Contents

Front Matter	1
1.1 Executive Definition	1
Research Foundation	5
2.1 Research Problem	5
2.2 Research Questions	5
2.3 Hypotheses	6
2.4 Expected Contributions	6
2.5 Novelty And Validation Strategy	6
2.6 Related Work And Evidence Map	7
Product Definition	8
3.1 Executive Definition	8
3.2 Product Rationale	8
3.2.1 Extended Rationale	8
3.3 Product Design	10
3.3.1 Product Scope	10
3.3.2 What MADRE Is	10
3.3.3 What MADRE Is Not	10
3.3.4 Core User Value	11
3.3.5 Design Principles	11
3.3.6 Product Boundaries	11

Front Matter

Abstract

1.1 Executive Definition

"MADRE is a local-first, model-agnostic runtime kernel for building agentic systems that preserve user sovereignty while coordinating reasoning modules, module-owned agents, governed context, bounded actions, known material, evaluated learning, recovery, and inspectable execution."

"

MADRE Agentic System[Beta v0.1] exists because the current AI market is normalizing a dangerous bargain: users receive convenient language interfaces, while their data, dependency chain, costs, and decision power move into remote platforms they do not control. That bargain is technically unnecessary and politically unacceptable for many individuals, teams, public-interest organizations, researchers, creators, and small developers.

The project does not treat a language model as a complete intelligence. A model is a powerful linguistic function: useful, fallible, statistically trained, expensive in context, and unsafe when it is allowed to become the whole system. **MADRE** puts that function inside an engineered runtime. The runtime decides what may be known, what may be sent, what may be stored, what may be acted on, and what must wait for stronger evidence or human authority.

The product objective is one coherent assistant experience backed by a durable kernel. Visible interaction can remain fast and conversational while module-local reasoning and reflection lanes decompose, schedule, check, record, and resume deeper work. This is the practical meaning of Model Agnostic Delayed Reasoning Effort: useful local intelligence should come from architecture, context discipline, and bounded autonomy, not from blind dependence on one remote provider or one large model.

MADRE is therefore not a chatbot, not a prompt collection, not a wrapper around one vendor API, and not a task-management application. It is the runtime layer that applications use to coordinate models, internal actions, conversation and knowledge records, policies, and interfaces without surrendering the user's private domain to cloud-first AI infrastructure.

Specification Scope

MADRE is a local-first, model-agnostic runtime kernel for constructing agentic systems whose useful behavior is produced by software architecture rather than by treating language models as autonomous intelligence. The runtime assigns a Core Module, permits its Main Agent role to provide foreground continuity through a module-owned **SystemAgent**, governs context before **ModelAction** or other actions,

keeps **KnowledgeRecord** handling inspectable, records local traces through **RuntimeJournal**, and requires learning to pass through evaluation, promotion, and rollback boundaries.

This specification defines the research, product, and architecture baseline for **MADRE**. It states the system problem, expected contribution, stakeholder concerns, requirement domains, architecture views, safety model, policy model, and evaluation surface used to assess runtime behavior. The product is an open, extensible foundation for private and affordable agentic software on ordinary local devices, with optional remote services and integrations treated as governed risk boundaries rather than defaults.

The scope of **MADRE** comprises runtime purpose, system boundary, requirement families, architecture obligations, risk model, quality attributes, validation strategy, and product-level authority semantics.

The design identifiers D-001 through D-013 are internal traceability anchors inside this dossier. They identify product rationale, runtime obligations, safety boundaries, and evaluation surfaces at architecture level without making prior side specifications authoritative.

Canonical Terms

Term	Meaning in MADRE
MADREKernel	Deterministic local coordinator that registers ReasoningModule objects, assigns the Core Module, routes requests, schedules work, coordinates resources, applies policy, and records runtime activity through RuntimeJournal .
ReasoningModule	Governed reasoning unit for a bounded domain or scope. It owns module records, workflows, routines, actions, agents, knowledge and learning boundaries, and allowed model/inference scope.
Core Module	Kernel registration role held by a compatible ReasoningModule that receives ordinary interaction first; it is not a separate module type.
MADREAgent / Main Agent role	A MADREAgent is a concrete module-owned runtime actor constrained by ModelBinding ; Main Agent role is the role held by one module-owned ComplexAgent , while other ComplexAgent objects need not hold that role.

Term	Meaning in MADRE
SimpleAgent, TwinAgent, ComplexAgent	Agent profiles for direct execution, review/evaluation over a produced result, and foreground/reasoning/reflection capability respectively; none gains authority outside its owning module.
ReasoningPlan, Workflow, AgentRoutine	A request-derived reasoning plan may select reusable module workflow and agent routine capabilities without turning either into a second plan or a new authority layer.
Access surface	A CLI, local console, GUI, speech surface, or application interface that exposes the kernel without becoming runtime authority.
ReasoningTask, RuntimeEvent, RuntimeJournal	ReasoningTask tracks delayed-work lifecycle; RuntimeEvent is append-only runtime history; RuntimeJournal is the local trace mechanism that stores or references them and related payloads.
PolicyDecision	Explicit allow, block, or authorization-required outcome for an affected action or data class and authority boundary.
Foreground lane	A possible ComplexAgent capability used by a module's Main Agent role to preserve flow, clarify intent, answer within safe context, and create delayed work when stronger evidence is required.
Reasoning lane	Durable module-local work used for analysis, evidence gathering, verification, internal action invocation, or higher-effort inference.
source material / ContextBundle	Source descriptor and bounded, classified, minimized, revocable selection of material prepared for an agent request, action, or inference execution.

Term	Meaning in MADRE
AgentAction, ModelAction, ModelBinding, model-use profile, model backend	An executable capability and its inference-backed specialization; a model-backed action executes only through the binding allowed for its module-owned agent, with configured and replaceable inference execution.
RuntimeEvent evidence / generated material	One recorded inference execution and its generated material. generated material is not truth, policy, memory, knowledge, behavior, routing, authority, or a candidate by itself.
KnowledgeRecord / KnowledgeCandidate	Known material held by a module with provenance, scope, sensitivity, intended use, truth-level/authority, correction, and lifecycle metadata; only a pending knowledge-boundary change is a KnowledgeCandidate.
LearningCandidate / retained interaction material	An evaluated proposed module improvement and retained interaction material respectively. Saving known material is not learning, and conversation is not knowledge by default.
Artifact	A materialized output such as a file, document, code patch, report, image, dataset, PDF, snapshot, or final buffer/document. Ordinary conversation text is not an artifact unless materialized.

Specification Baseline

Version	Status
Beta v0.1	Research, product, and architecture specification baseline for local-first, model-agnostic delayed reasoning. The baseline covers product requirements, architecture obligations, safety boundaries, traceability, and evaluation criteria.

Research Foundation

2.1 Research Problem

Modern language-model systems often combine private data, model calls, action execution, retained interaction, known material, and remote infrastructure inside opaque conversational interfaces. This makes useful demonstrations easy to produce, but it weakens local ownership, traceability, autonomy control, error recovery, and the ability to replace models or providers.

MADRE addresses this as a Software Engineering problem: how to design a local-first runtime in which language models remain fallible replaceable functions, while the software system owns policy, context, knowledge, internal actions, runtime trace, recovery, and learning boundaries. The research problem is not to prove that models reason, but to define an architecture that extracts useful language behavior without surrendering system authority to generated material.

2.2 Research Questions

ID	Research question
RQ-1	How can a local-first runtime separate immediate interaction from delayed reasoning without losing continuity, accountability, or user control?
RQ-2	How can model-agnostic contracts prevent provider-specific assumptions from entering policy, knowledge handling, action authority, and truth management?
RQ-3	What traceability structure is sufficient to connect MADRE 's product claims, stakeholder concerns, runtime requirements, architecture decisions, and validation evidence?
RQ-4	How can context, retained interaction material, KnowledgeRecord , KnowledgeCandidate , and LearningCandidate remain useful while preserving provenance, truth authority, minimization, revocation, correction, promotion, and rollback boundaries?
RQ-5	What integrated behavior demonstrates that MADRE is a governed runtime rather than a conversational wrapper?

2.3 Hypotheses

ID	Hypothesis
H-1	Module-local foreground, reasoning, and reflection lanes can preserve conversational responsiveness while improving reliability for tasks that require evidence, verification, action execution, or stronger context.
H-2	Model-agnostic contracts can reduce provider lock-in when policy, knowledge handling, context, and action execution remain owned by the runtime.
H-3	Context governance with provenance, classification, minimization, and revocation can reduce leakage and misuse without eliminating useful specialized reasoning.
H-4	Evaluated LearningCandidate promotion can improve system adaptation while preventing generated material or ordinary KnowledgeRecord storage from silently rewriting protected behavior or factual authority.
H-5	RuntimeJournal , ReasoningTask , and append-only RuntimeEvent history can make delayed reasoning inspectable and recoverable enough for ordinary local operation.

2.4 Expected Contributions

MADRE’s expected contribution is a reference architecture for local-first, model-agnostic delayed reasoning. The contribution combines module-owned agents, optional Complex Agent lane profiles, governed context bundles, explicit **ModelAction** and internal action contracts, **RuntimeEvent** evidence/generated material separation, controlled learning, and traceable recovery paths.

The novelty is not a new language model. It is the engineering arrangement that keeps language models useful while preventing them from becoming sources of truth, authority, policy, knowledge, or action. The architecture is intended to be useful for personal assistants, developer systems, organizational knowledge workers, domain systems, and local interfaces that require control over private knowledge.

2.5 Novelty And Validation Strategy

The first research claim is that a useful agentic system can be validated as a software architecture before it is validated as a broad automation product. **MADRE**’s novelty is the conjunction of local-first execution, replaceable inference and action boundaries, delayed reasoning, governed context, explicit **PolicyDecision** outcomes, evaluated learning promotion, and journaled recovery.

The validation strategy examines whether a local request can enter through bounded access surfaces, be triaged without unsupported claims, become a durable **ReasoningTask** when stronger evidence is needed, obtain a governed context bundle, cross a replaceable model backend, invoke a bounded internal action after a permitting **PolicyDecision**, produce an generated material that remains non-authoritative, expose **RuntimeJournal** trace evidence, and support correction, invalidation, supersession, rollback, or deletion/detachment.

The validation strategy treats negative paths as first-class evidence. Prompt or source injection must remain source data. Known material used with an incorrect **truthLevel** or **truthAuthority** must be rejected or corrected. Sensitive or unrelated material must be excluded from context. generated

material must not become truth, policy, action authority, memory, knowledge, behavior, routing, or a candidate by itself. A **KnowledgeCandidate** changes a knowledge boundary only through promotion, and a **LearningCandidate** changes module behavior only after defined evaluation and promotion.

2.6 Related Work And Evidence Map

Evidence family	Relevance to MADRE	Specification use
Local-first software	Establishes user ownership, offline usefulness, privacy, longevity, and optional synchronization as product concerns.	Supports data sovereignty and local-first architecture claims.
Requirements engineering	Provides structure for measurable requirements, acceptance criteria, verification methods, and source traceability.	Supports the SRS and traceability model.
Architecture description	Provides stakeholder, concern, viewpoint, view, correspondence, and decision-rationale concepts.	Supports multi-view architecture description.
LLM and agent security	Identifies prompt injection, sensitive disclosure, supply-chain risk, data poisoning, excessive agency, vector weakness, misinformation, and resource consumption.	Supports threat and policy modeling.
Software quality models	Provide vocabulary for reliability, maintainability, portability, security, performance efficiency, usability, and quality-in-use evaluation.	Supports the quality attribute model.
AI evaluation research	Provides multi-metric and evidence-oriented evaluation patterns for generated material, retrieval quality, and agentic task performance.	Supports MADRE-native benchmark families.

Product Definition

3.1 Executive Definition

MADRE is a runtime foundation rather than a single assistant interface. The kernel assigns ordinary interaction to a Core Module, records delayed lifecycle and runtime history locally, and keeps policy, knowledge, action, and recovery authority outside inference. Language models run through module-constrained **ModelAction** execution; each generated material is generated material from an **RuntimeEvent** evidence, not authority or a candidate by itself.

3.2 Product Rationale

MADRE Agentic System

Rationale	Product implication
Local control	Private data, local knowledge, and sensitive work remain under local authority unless a policy-governed transfer is explicitly permitted.
Model substitutability	No model provider, host, prompt convention, or output format defines runtime truth, policy, knowledge, or action authority.
Delayed reasoning	Work that requires evidence, verification, resource control, or stronger context is represented as a traceable ReasoningTask rather than as an unsupported foreground answer.
Context governance	Context is selected, minimized, classified, sourced, scoped, provenanced, and revocable before it is provided to a ModelAction , internal action, or reasoning task.
Bounded action use	Internal actions, external actions, knowledge change, and workflow execution are allowed only through declared authority boundaries, PolicyDecision outcomes, trace evidence, and recovery expectations.
Known material and learning	A KnowledgeRecord is not factual truth by default; a KnowledgeCandidate is reserved for pending knowledge-boundary change, while a LearningCandidate is an evaluated proposed module improvement.

3.2.1 Extended Rationale

- **Data Sovereignty:**

The first design obligation of **MADRE** is to keep the user's data under the user's control. Contemporary AI services often make private life, professional work, organizational knowledge,

and creative material flow through remote systems by default. That default is not neutral. It concentrates power in the companies that own the infrastructure, the logs, the model services, the policy switches, and the billing relation.

MADRE refuses to make surveillance a prerequisite for assistance. Local-first operation is a security boundary, a political boundary, and an engineering discipline. Private conversation, module-owned known material, organizational records, and sensitive work must remain local unless an explicit policy, classification, minimization, and trace path says otherwise.

Internet access is treated as a risk boundary. Prompt injection, credential exposure, external action abuse, provider lock-in, and remote data retention are normal failure modes of cloud-first agentic systems; **MADRE** 's architecture must make those failures harder by default and visible when they cannot be eliminated.

- **Free Software Duty:**

MADRE is released under GPL-3.0 because source availability alone is not enough. The project aims to provide usable agentic infrastructure for people and organizations that cannot, or should not, depend on platform owners for every intelligent interface they build.

The current generation of AI models draws on a broad inheritance of human language, labor, research, art, software, and social knowledge. A public-interest response requires practical, inspectable infrastructure outside exclusive corporate channels.

MADRE supports developers and creators running useful systems on ordinary machines, inspecting decisions, replacing inference backends and authorized integrations, and sharing improvements without requiring platform permission.

- **Model Agnosticism:**

No single model, host, vendor, format, or prompt convention should define **MADRE** 's architecture. Models improve, fail, disappear, become expensive, change license terms, and carry different bias and safety profiles.

MADRE treats inference as a governed **ModelAction**. The runtime may choose different permitted model-use profile and model backend combinations without letting any model become authority over policy, knowledge, actions, or truth.

- **Resource Discipline:**

Local-first AI must remain useful on ordinary commercial devices and degrade intelligently when resources are scarce.

Optional Complex Agent lanes, task decomposition, caching, context minimization, repeated checks where they matter, and durable records make stronger reasoning selective rather than an unlimited default.

- **Knowledge Governance:**

Agentic systems become dangerous when they blur text, known material, retained interaction, instruction, evidence, and action. A conversation turn is not a verified fact, an generated material is not a source or authority, and an action result is not an instruction.

A **KnowledgeRecord** carries provenance, sensitivity, intended use, **truthLevel**, and **truthAuthority**; a **LearningCandidate** is an evaluated module-improvement proposal, not ordinary stored knowledge.

- **Distributed Reasoning:**

Interaction can remain coherent while module-owned agents defer hard work, retrieve permitted material, test hypotheses, revise plans, and learn from evaluated outcomes without treating language models as minds.

Every agent, `RuntimeEvent` evidence, action invocation, `KnowledgeRecord` change, and `LearningCandidate` must remain accountable to an explicit runtime boundary.

3.3 Product Design

3.3.1 Product Scope

MADRE provides a disciplined coordination layer underneath agentic applications. The kernel routes interaction, registers the Core Module, records delayed work through `RuntimeJournal`, coordinates bounded context and actions, records `PolicyDecision` outcomes, and exposes recovery semantics.

The primary audience is developers and organizations building private, domain-aware, extensible assistant systems. Applications built on **MADRE** may vary in interface, domain, and policy, but they inherit the same authority model: modules own their agents and known material, model execution is constrained by `ModelBinding`, and policy, trace, action, and recovery boundaries remain runtime-controlled.

3.3.2 What MADRE Is

- A local-first runtime kernel for agentic systems.
- A model-agnostic coordination layer for multiple model families.
- A delayed-reasoning architecture that keeps conversation responsive while deeper work happens through module-local agents and `ReasoningTask` lifecycle.
- A policy-aware boundary for context, internal actions, knowledge, learning, and remote transfer.
- A developer foundation for building assistants, domain systems, and workflow-capable applications on top of governed local intelligence.

3.3.3 What MADRE Is Not

- **MADRE** is not a chatbot product. Chat is one possible surface over the kernel, not the kernel's definition.
- **MADRE** is not a prompt wrapper. Prompts are runtime inputs, not an architecture.
- **MADRE** is not a vendor abstraction layer whose purpose is merely to swap API providers.
- **MADRE** is not an unrestricted automation agent. Internal actions, external actions, knowledge-boundary change, and learning require explicit boundaries.
- **MADRE** is not a passive record of interaction history. The product is the governed runtime kernel and its architectural obligations.

3.3.4 Core User Value

MADRE lets an application answer quickly without pretending every answer can be safe, complete, or well-reasoned in the foreground. The Core Module can assign a Main Agent role with a Complex Agent profile whose foreground lane clarifies intent, uses appropriately classified material, names uncertainty, and triggers deeper work. Module-local reasoning lanes perform work requiring decomposition, evidence, internal actions, verification, or stronger context.

This separation gives users a better contract. They are not forced to choose between a quick hallucination and a slow black box. They get immediate conversation when that is appropriate, delayed reasoning when that is necessary, and inspectable boundaries when the system touches sensitive data or meaningful actions.

3.3.5 Design Principles

- **Local-first, not local-only:** Local execution is the default because privacy, cost, resilience, and agency require a strong local base. Remote services may exist only as explicitly governed integrations with classification, minimization, policy, provenance, and journaled trace.
- **Model-agnostic by contract:** Models are replaceable runtime functions. The kernel must not let any model provider define the system's truth model, policy boundary, knowledge representation, or action authority.
- **Foreground lane, durable reasoning:** A Main Agent role may use a Complex Agent foreground lane to preserve conversational flow. Expensive, uncertain, risky, or evidence-heavy work should become a scheduled `ReasoningTask` that can be resumed, inspected, verified, and delivered asynchronously.
- **Context is a governed asset:** Context is not just text stuffed into a prompt. It is selected, minimized, classified, sourced, scoped, and revoked according to the purpose of the task and the authority of the actor using it.
- **Autonomy is bounded:** **MADRE** must support unattended delayed work, but autonomy is legitimate only when the action has an applicable `PolicyDecision`, is recorded through `RuntimeEvent`, is recoverable for its risk level, and is blocked from crossing stronger boundaries without authorization.
- **Learning is evaluated:** Useful systems should improve from use, but a `LearningCandidate` cannot quietly rewrite protected behavior. Saving a `KnowledgeRecord` is not learning; knowledge-boundary and behavior changes must be reviewable, promotable, reversible, and separable.
- **Interfaces do not own the architecture:** The `madre` CLI, local read-first inspection console, graphical UI, speech surfaces, or domain applications are access surfaces. They must expose and operate the kernel without becoming the source of runtime authority.

3.3.6 Product Boundaries

The product boundary covers runtime responsibilities and architecture obligations: local-first operation, model-agnostic inference, module-owned agent coordination, governed context, bounded internal actions, knowledge handling, evaluated learning, runtime traceability, and recovery.

MADRE may expose many application forms: personal assistants, research systems, developer environments, organizational knowledge workers, or domain-specific automation environments. Those applications may differ in interface and policy, but they must inherit the same kernel commitments: local-first operation, replaceable inference, governed context, bounded internal actions, inspectable retained interaction material and **KnowledgeRecord** handling, evaluated learning, journaled runtime history, and recovery.